Flutter Day 2 - Delegate Handbook

Widgets & Layout · MO Integrations · Flutter Mobile Application Development

> **How to use this document.** Open it in VS Code alongside the course so you can copy blocks straight out of it. Every code block is complete as it stands. Windows commands come first because that is what the room runs; macOS follows each one.
>
> **Today's deliverable:** `quote_app` gains a brand header, a responsive shell, a fully validated capture form, and two switchable brand themes. All of it survives into Days 3, 4 and 5.

Part 0 · Where we are

You finished Day 1 with:

File	What is in it
`lib/main.dart`	`runApp(const QuoteApp())`
`lib/app.dart`	`QuoteApp`, a `StatelessWidget` returning `MaterialApp`
`lib/features/quote/domain/quote_model.dart`	`Cover`, `QuoteRequest`, `Quote`, `Money` extension, `calculatePremium`, `QuoteState`
`lib/features/quote/presentation/capture_screen.dart`	`CaptureScreen` (Stateful), `PremiumBadge`

Everything today imports from `quote_model.dart`. There is only that one domain file, so every import in this handbook points at it.

If your Day 1 code is broken or missing, do not debug it during a lab. Clone the course repo and take the Day 1 end state from there:

```
git clone https://github.com/Clynton19860/flutter.git
```

Then copy the Day 1 solution files into your own `quote_app`.

Reminder from yesterday: Windows PowerShell 5.1 has no `&&`. Every command in this document is one line at a time. Run them one at a time.

Part 1 · The one rule: constraints

This is the whole layout system in three sentences.

> **Constraints go down. Sizes go up. The parent sets the position.**

A parent tells a child "you may be between these minimum and maximum widths and heights". The child picks a size inside that range and reports it back. The parent then decides where to put it.

There are three kinds of constraint:

Kind	Meaning	Where you meet it
Tight	min equals max	`SizedBox(width: 100, height: 40)`, the `Scaffold` body
Loose	min is 0, max is some number	inside `Center`, `Align`, `Padding`
Unbounded	max is infinity	`ListView`, `SingleChildScrollView`, the main axis of `Row` and `Column`

Every layout error you hit today is one of three things: too tight, too loose, or unbounded.

```
Center(
  child: Container(
    color: Colors.amber,
  ),
)
```

A `Container` with no child and no size says "make me as big as I am allowed". Inside `Center` the constraints are loose but **bounded**, so it fills the whole screen.

```
Column(children: [
  Container(color: Colors.amber),
])
```

Same `Container`, inside a `Column`. The main axis of a `Column` is **unbounded**, so "as big as allowed" has no answer and the `Container` collapses to zero height. It vanishes.

```
SizedBox(width: 100, height: 40, child: Container(color: Colors.amber))
```

Tight constraints. Exactly 100 by 40, anywhere in the tree.

When you see red text or yellow and black stripes, ask one question: who gave whom which constraint?

Reference: docs.flutter.dev/ui/layout/constraints

Part 2 · Core widgets

2.1 Container

`Container` is a convenience wrapper around `Padding`, `Align`, `DecoratedBox` and `ConstrainedBox`.

```
Scaffold(
  body: Center(
    child: Container(
      width: 200,
      height: 120,
      padding: const EdgeInsets.all(16),
      margin: const EdgeInsets.symmetric(horizontal: 24),
      alignment: Alignment.bottomRight,
      decoration: BoxDecoration(
        color: Theme.of(context).colorScheme.primaryContainer,
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: Colors.black12),
        boxShadow: const [BoxShadow(blurRadius: 8, color: Colors.black26)],
      ),
      child: const Text('R 1 450 / month'),
    ),
  ),
)
```

Four things to remember:

1. `width` and `height` are **preferences**. Constraints from the parent win.
2. `color:` and `decoration:` together throws an assertion. Put the colour inside `BoxDecoration`.
3. `margin` is outside the decoration, `padding` is inside it.
4. `alignment:` makes the `Container` expand to the parent's maximum constraints. This surprises people when a small card suddenly fills the screen.

Prefer the specific widget when you only need one thing: `Padding`, `SizedBox`, `DecoratedBox`, `Align`. They are cheaper and they say what they mean.

2.2 Text, Icon and Image

```
Text(
  'Comprehensive cover',
  style: Theme.of(context).textTheme.titleMedium,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)

Text.rich(TextSpan(children: [
  const TextSpan(text: 'Premium '),
  TextSpan(text: 'R 1 450', style: Theme.of(context).textTheme.titleLarge),
]))

Icon(Icons.shield_outlined, color: Theme.of(context).colorScheme.primary, size: 32)

Image.asset('assets/brands/alpha.png', height: 40, semanticLabel: 'Alpha Insure logo')

Image.network(
  'https://example.com/hero.jpg',
  fit: BoxFit.cover,
  loadingBuilder: (ctx, child, progress) =>
      progress == null ? child : const CircularProgressIndicator(),
  errorBuilder: (ctx, err, st) => const Icon(Icons.broken_image),
)
```

`overflow: TextOverflow.ellipsis` only works if the `Text` has a bounded width. Inside a `Row` that means wrapping it in `Expanded`.

2.3 The Material 3 button family

Widget	Use it for	Notes
`FilledButton`	The primary action, one per screen	M3 default. `.icon` variant, `.tonal` for secondary emphasis
`ElevatedButton`	Legacy primary look	Still fine, but prefer `FilledButton` in M3
`OutlinedButton`	Secondary action	Same API as `FilledButton`
`TextButton`	Low emphasis: dialog actions, links	
`IconButton`	Icon only, toolbar or app bar	Always set `tooltip:`
`FloatingActionButton`	One screen level action	Goes in `Scaffold.floatingActionButton`
`SegmentedButton<T>`	Pick one or many of two to five options	Replaces `ToggleButtons` in M3

To disable any button, pass `onPressed: null`. It greys out automatically.

On our capture screen that maps to: **Get quote** is a `FilledButton`, **Reset** is an `OutlinedButton`, **Terms** would be a `TextButton`.

2.4 Stateless or Stateful

Make it Stateless when	Make it Stateful when
Everything it shows comes from constructor arguments	It owns a controller: `TextEditingController`, `ScrollController`, `FocusNode`, `AnimationController`
It reacts to nothing except a rebuild from its parent	It has local UI state nobody else needs: a toggle, expanded or collapsed, the current tab
Today: `_BrandHeader`, `PremiumBadge`, quote tiles, most cards	It needs `initState` or `dispose`
This is the default choice	Today: `CaptureForm`, and the brand toggle until Riverpod replaces it tomorrow

Business state (the quote, the user, the signed-in account) is neither. It goes to Riverpod tomorrow.

The two widget test: if two widgets need the same value, it does not belong inside either of them.

2.5 Three habits

1. Extract Widget. Put the cursor on a subtree, press `Ctrl+.` (macOS `Cmd+.`), choose **Extract Widget**, name it.

Prefer a new class over a private `Widget _buildHeader()` method. A class gets its own element, can be `const`, and can skip rebuilds. A method rebuilds every time its parent rebuilds and can never be `const`.

2. const constructors. A `const` widget is created once and skipped on every rebuild. The analyser suggests them and this applies them across the project:

```
dart fix --dry-run
```

```
dart fix --apply
```

```
dart format .
```

Those three are identical on Windows and macOS.

3. Keys. You only need them when siblings of the same type get reordered, inserted or removed: lists, `Dismissible`, `AnimatedList`. Use `ValueKey(item.id)`, never the index.

> **Rule of thumb.** If a `build()` method does not fit on one screen, extract. If a widget takes no arguments that change, make it `const`.

Part 3 · The layout system

3.1 Row and Column

```
Column(
  mainAxisAlignment: MainAxisAlignment.start,
  crossAxisAlignment: CrossAxisAlignment.stretch,
  mainAxisSize: MainAxisSize.min,
  children: [
    Row(
      children: [
        const Icon(Icons.directions_car),
        const SizedBox(width: 8),
        Expanded(child: Text('VW Polo 2020', overflow: TextOverflow.ellipsis)),
        Text('R 1 450', style: Theme.of(context).textTheme.titleMedium),
      ],
    ),
    const SizedBox(height: 12),
    Row(
      children: [
        Expanded(flex: 2, child: FilledButton(onPressed: () {}, child: const Text('Accept'))),
        const SizedBox(width: 8),
        Expanded(flex: 1, child: OutlinedButton(onPressed: () {}, child: const Text('Edit'))),
      ],
    ),
  ],
)
```

Property	What it does
main axis	`Row` horizontal, `Column` vertical. `mainAxisAlignment` spaces children along it
cross axis	the other one. `crossAxisAlignment`: start, center, end, stretch
`mainAxisSize`	`max` fills the parent (the default), `min` shrink wraps
`Expanded`	the child takes **exactly** its share of the leftover space
`Flexible`	the child takes **up to** its share
`Spacer` / `SizedBox`	a flexible empty gap / a fixed gap

`CrossAxisAlignment.stretch` on a `Column` makes buttons full width. That is the pattern for every form in this course.

Coming from CSS flexbox: `Row` is `flex-direction: row`, `mainAxisAlignment` is `justify-content`, `crossAxisAlignment` is `align-items`, `Expanded` is `flex: 1`. The one real difference: a `Row` child with no `Expanded` gets **unbounded** width, which is why a long `Text` overflows unless you wrap it.

3.2 The four errors you will see today

```
// 1) "A RenderFlex overflowed by 42 pixels on the right"
Row(children: [Text('A very long vehicle description that will not fit')])
// FIX: give the text bounded width
Row(children: [Expanded(child: Text('A very long…', overflow: TextOverflow.ellipsis))])

// 2) "Vertical viewport was given unbounded height"
Column(children: [ListView(children: const [Text('a')])])
// FIX: bound the ListView
Column(children: [Expanded(child: ListView(children: const [Text('a')]))])
// short lists only: ListView(shrinkWrap: true, physics: NeverScrollableScrollPhysics())

// 3) "Incorrect use of ParentDataWidget"
Container(child: Expanded(child: Text('x')))
// FIX: Expanded and Flexible only go directly inside Row, Column or Flex
Column(children: [Expanded(child: Text('x'))])

// 4) "BoxConstraints forces an infinite width"
Row(children: [TextField()])
// FIX
Row(children: [Expanded(child: TextField())])
```

Read the **first line** of the red error text. It names the widget and the axis. Then ask who handed it unbounded or too tight constraints.

`shrinkWrap: true` lays out every child, so it is O(n). Use it for a handful of rows, never for a long list.

3.3 Stack and Positioned

```
Stack(
  clipBehavior: Clip.none,
  children: [
    Container(
      height: 160,
      decoration: BoxDecoration(
        color: Theme.of(context).colorScheme.primary,
        borderRadius: BorderRadius.circular(16),
      ),
    ),
    Positioned(
      left: 16, top: 16,
      child: Text('Alpha Insure',
          style: Theme.of(context).textTheme.titleLarge?.copyWith(color: Colors.white)),
    ),
    Positioned(
      right: 16, bottom: -20,
      child: Chip(label: const Text('Comprehensive'), backgroundColor: Colors.white),
    ),
    const Positioned.fill(
      child: Align(
        alignment: Alignment.center,
        child: Icon(Icons.shield, size: 48, color: Colors.white24),
      ),
    ),
  ],
)
```

- Children are painted in order. Later ones sit on top.
- Non positioned children are placed by the Stack's `alignment:`.
- `Positioned.fill` covers the whole Stack. Use it for overlays, scrims and full area tap targets.
- Negative offsets need `clipBehavior: Clip.none`, otherwise the overhang is clipped away.
- **A Stack whose children are all `Positioned` has no size.** Give it a sized child or use `fit: StackFit.expand`.

Coming from CSS: `Stack` is `position: relative`, `Positioned` is `position: absolute`.

3.4 Responsive layout

```
// 1) Screen size
final size = MediaQuery.sizeOf(context);
final isWide = size.width >= 600;

// 2) Parent constraints, which is what you want inside a layout
LayoutBuilder(
  builder: (context, constraints) {
    final twoColumn = constraints.maxWidth >= 600;
    return twoColumn
        ? Row(children: [Expanded(child: form), Expanded(child: summary)])
        : Column(children: [form, summary]);
  },
)

// 3) Orientation
OrientationBuilder(
  builder: (context, o) => o == Orientation.portrait ? portrait : landscape,
)

// 4) Notch, status bar, gesture bar
SafeArea(child: body)

// 5) Text scaling
final scale = MediaQuery.textScalerOf(context);
```

`MediaQuery` gives you the **screen**. `LayoutBuilder` gives you **your slot**. Inside a `Row`, a dialog or a split pane, the screen size is the wrong number.

Use `MediaQuery.sizeOf(context)` rather than `MediaQuery.of(context).size`. The first rebuilds only when the size changes; the second rebuilds when anything in `MediaQuery` changes, including the keyboard appearing.

Material breakpoints: compact below 600, medium 600 to 840, expanded 840 and up. Our capture screen goes two column at 700.

`SafeArea` goes **inside** the `Scaffold` body, not around the `Scaffold`.

Users set 130 per cent font scaling. Never fix a height around text. On Day 5 we test at 200 per cent.

LAB 2.1 · Brand header and responsive shell · 35 min

Goal: `CaptureScreen` has a brand header and switches to a two column layout above 700 px, with no overflow at any width.

Steps

1. Open `lib/features/quote/domain/quote_model.dart` and add a `label` getter to the `Cover` enum (code below).
2. Rewrite `capture_screen.dart`: `CaptureScreen` becomes a `StatelessWidget` with a `_BrandHeader` and a placeholder card where the form will go.
3. Upgrade `_BrandHeader` to the `Stack` version from Part 3.3: a coloured card, the title positioned top left, and a `Chip` hanging off the bottom right edge with `clipBehavior: Clip.none`.

4. Wrap the body in `SafeArea` then `LayoutBuilder`. Below 700 px use a scrolling `Column`. At 700 and above use a `Row` with a fixed width header and an expanded scrolling form area.
5. Rotate the emulator to landscape and confirm the layout switches.
6. Deliberately put a `ListView` inside the `Column`, read the error, fix it with `Expanded`, then remove it again.
7. Open DevTools, go to Flutter Inspector, open the **Layout Explorer**, select the `Row` and look at the flex factors.
8. Run the checks and commit.

Step 1 code: add `Cover.label`

Replace the `Cover` enum at the top of `quote_model.dart` with this:

```
enum Cover {
  thirdParty, thirdPartyFireTheft, comprehensive;

  double get factor => switch (this) {
        Cover.thirdParty => 0.6,
        Cover.thirdPartyFireTheft => 0.8,
        Cover.comprehensive => 1.0,
      };

  String get label => switch (this) {
        Cover.thirdParty => 'Third party',
        Cover.thirdPartyFireTheft => 'Third party, fire & theft',
        Cover.comprehensive => 'Comprehensive',
      };
}
```

Step 8 commands

```
flutter analyze
```

```
dart fix --apply
```

```
git add .
```

```
git commit -m "Day 2: header and responsive shell"
```

Solution: `lib/features/quote/presentation/capture_screen.dart`

This is the state of the file at the end of Lab 2.1. Lab 2.2 replaces the placeholder card with the real form.

```dart
import 'package:flutter/material.dart';
import 'package:quote_app/features/quote/domain/quote_model.dart';

class CaptureScreen extends StatelessWidget {
  const CaptureScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Get a quote')),
      body: SafeArea(
        child: LayoutBuilder(
          builder: (context, c) {
            const header = _BrandHeader(name: 'Alpha Insure');
            const body = Padding(
              padding: EdgeInsets.all(16),
              child: Card(
                child: Padding(
                  padding: EdgeInsets.all(24),
                  child: Text('Capture form goes here in Lab 2.2'),
                ),
              ),
            );

            if (c.maxWidth >= 700) {
              return const Row(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  SizedBox(width: 280, child: header),
                  Expanded(child: SingleChildScrollView(child: body)),
                ],
              );
            }
            return const SingleChildScrollView(
              child: Column(children: [header, body]),
            );
          },
        ),
      ),
    );
  }
}

class _BrandHeader extends StatelessWidget {
  const _BrandHeader({required this.name});
  final String name;

  @override
  Widget build(BuildContext context) {
    final cs = Theme.of(context).colorScheme;
    final tt = Theme.of(context).textTheme;
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 16, 16, 32),
      child: Stack(
        clipBehavior: Clip.none,
```

```
        children: [
          Container(
            width: double.infinity,
            height: 160,
            decoration: BoxDecoration(
              color: cs.primaryContainer,
              borderRadius: BorderRadius.circular(16),
            ),
          ),
          const Positioned.fill(
            child: Align(
              alignment: Alignment.center,
              child: Icon(Icons.shield, size: 64, color: Colors.white24),
            ),
          ),
          Positioned(
            left: 16,
            top: 16,
            right: 16,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(name,
                    style: tt.titleLarge?.copyWith(color: cs.onPrimaryContainer)),
                const SizedBox(height: 4),
                Text('Get a motor quote in under a minute',
                    style: tt.bodyMedium?.copyWith(color: cs.onPrimaryContainer)),
              ],
            ),
          ),
          Positioned(
            right: 16,
            bottom: -16,
            child: Chip(
              label: const Text('Comprehensive'),
              backgroundColor: cs.surface,
            ),
          ),
        ],
      ),
    );
  }
}

class PremiumBadge extends StatelessWidget {
  const PremiumBadge({super.key, required this.amount});
  final double amount;

  @override
  Widget build(BuildContext context) => Text(
        amount.rands,
        style: Theme.of(context).textTheme.headlineMedium,
      );
}
```

Stretch

- Add three cover `Chip`s under the tagline using `Wrap` so they reflow at narrow widths.
- Collapse the header to a single row when `constraints.maxHeight < 500`, which is a phone in landscape.

Part 4 · Lists

4.1 ListView

```
ListView.separated(
  padding: const EdgeInsets.all(16),
  itemCount: quotes.length,
  separatorBuilder: (_, __) => const SizedBox(height: 8),
  itemBuilder: (context, i) {
    final q = quotes[i];
    return Dismissible(
      key: ValueKey(q.id),
      direction: DismissDirection.endToStart,
      background: Container(
        alignment: Alignment.centerRight,
        padding: const EdgeInsets.only(right: 16),
        color: Theme.of(context).colorScheme.error,
        child: const Icon(Icons.delete, color: Colors.white),
      ),
      onDismissed: (_) => onDelete(q),
      child: Card(
        child: ListTile(
          leading: const Icon(Icons.description_outlined),
          title: Text(q.display),
          subtitle: Text('Quote ${q.id}'),
          trailing: const Icon(Icons.chevron_right),
          onTap: () {},
        ),
      ),
    );
  },
)
```

Constructor	When
`ListView(children: [...])`	Builds everything up front. Fine for fewer than about 20 fixed items
`ListView.builder`	Builds only visible rows. Use for anything data driven
`ListView.separated`	Same, plus a `separatorBuilder`. Cleaner than a `SizedBox` inside each row

The `Dismissible` trap. If you do not remove the item from the underlying list inside `onDismissed`, Flutter throws *"A dismissed Dismissible widget is still part of the tree"*. The key must identify the **item**, so `ValueKey(q.id)`, never the index.

Always handle the empty list explicitly. Here is the full widget with its empty state, which we reuse on Day 4 with SQLite data:

```
class SavedQuotesList extends StatelessWidget {
  const SavedQuotesList({super.key, required this.quotes, required this.onDelete});
  final List<Quote> quotes;
  final void Function(Quote) onDelete;

  @override
  Widget build(BuildContext context) {
    if (quotes.isEmpty) {
      return const Center(child: Text('No saved quotes yet'));
    }
    return ListView.separated(
      padding: const EdgeInsets.all(16),
      itemCount: quotes.length,
      separatorBuilder: (_, __) => const SizedBox(height: 8),
      itemBuilder: (context, i) {
        final q = quotes[i];
        return Dismissible(
          key: ValueKey(q.id),
          direction: DismissDirection.endToStart,
          background: Container(
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 16),
            color: Theme.of(context).colorScheme.error,
            child: const Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (_) => onDelete(q),
          child: Card(
            child: ListTile(
              leading: const Icon(Icons.description_outlined),
              title: Text(q.display),
              subtitle: Text('Quote ${q.id}'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {},
            ),
          ),
        );
      },
    );
  }
}
```

4.2 GridView

```
GridView.builder(
  padding: const EdgeInsets.all(16),
  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
    maxCrossAxisExtent: 220,
    mainAxisSpacing: 12,
    crossAxisSpacing: 12,
    childAspectRatio: 1.2,
  ),
  itemCount: Cover.values.length,
  itemBuilder: (context, i) {
    final c = Cover.values[i];
    return Card(
      child: InkWell(
        onTap: () => onSelect(c),
        child: Padding(
          padding: const EdgeInsets.all(12),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Icon(Icons.shield_outlined),
              const Spacer(),
              Text(c.label, style: Theme.of(context).textTheme.titleMedium),
              Text('× ${c.factor}'),
            ],
          ),
        ),
      ),
    );
  },
)
```

`SliverGridDelegateWithMaxCrossAxisExtent` means "as many columns of at most 220 px as will fit". A phone gets one or two, a tablet gets three or four, and you write no breakpoint code at all. The alternative, `SliverGridDelegateWithFixedCrossAxisCount`, pins the column count.

4.3 Scrolling helpers

```
RefreshIndicator(
  onRefresh: () async => await reload(),
  child: ListView.builder(
    physics: const AlwaysScrollableScrollPhysics(),
    itemCount: items.length,
    itemBuilder: (_, i) => ListTile(title: Text(items[i])),
  ),
)
```

```
final _scroll = ScrollController();

@override
void dispose() {
  _scroll.dispose();
  super.dispose();
}

_scroll.animateTo(0, duration: const Duration(milliseconds: 300), curve: Curves.easeOut);
```

`onRefresh` must return a `Future`. `AlwaysScrollableScrollPhysics` lets a short list still be pulled down.

`SingleChildScrollView` or `ListView`? One child that builds everything, versus lazy rows. Forms use the first. Data uses the second.

Part 5 · Forms

5.1 Form and controllers

```
class CaptureForm extends StatefulWidget {
  const CaptureForm({super.key, required this.onSubmit});
  final void Function(QuoteRequest) onSubmit;

  @override
  State<CaptureForm> createState() => _CaptureFormState();
}

class _CaptureFormState extends State<CaptureForm> {
  final _formKey = GlobalKey<FormState>();
  final _makeCtrl = TextEditingController();
  final _yearCtrl = TextEditingController();
  Cover _cover = Cover.comprehensive;
  DateTime? _licenceDate;

  @override
  void dispose() {
    _makeCtrl.dispose();
    _yearCtrl.dispose();
    super.dispose();
  }

  void _submit() {
    if (!_formKey.currentState!.validate()) return;
    final year = int.parse(_yearCtrl.text);
    final licenceYear = _licenceDate?.year ?? DateTime.now().year;
    final age = DateTime.now().year - licenceYear + 18;
    widget.onSubmit(QuoteRequest(
      make: _makeCtrl.text.trim(),
      year: year,
      driverAge: age,
      cover: _cover,
    ));
  }

  @override
  Widget build(BuildContext context) => Form(key: _formKey, child: _fields(context));
}
```

Piece	What it does
`Form`	Groups the fields. `GlobalKey<FormState>` is your handle to it
`validate()`	Runs every validator and displays the errors. Returns a bool
`save()`	Calls every `onSaved`
`reset()`	Clears values and errors
Validator return	`null` means valid. A `String` is the error shown under the field
`autovalidateMode`	`AutovalidateMode.onUserInteraction` validates as they type after the first attempt

Every `TextEditingController` and `FocusNode` must be disposed. If you do not, the debug build asserts and the app leaks. Write `dispose()` before you write `build()` so you cannot forget.

Create controllers in the State, never in `build()`. A controller created in `build()` is thrown away and rebuilt on every frame, which produces *"A TextEditingController was used after being disposed"*.

The `!` in `_formKey.currentState!` is justified here. The key is attached to a `Form` in this same build method, so it cannot be null when a button inside that form is tapped.

The callback goes up. `CaptureForm` does not know what happens to the request. Tomorrow that callback calls a Riverpod notifier instead, and nothing inside the form changes.

Coming from React Hook Form or Angular reactive forms: `Form` plus a `GlobalKey` is the form group, validators are per field functions, and controllers are the value accessors.

5.2 The fields

```
Widget _fields(BuildContext context) => Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    TextFormField(
      controller: _makeCtrl,
      decoration: const InputDecoration(labelText: 'Vehicle make', hintText: 'e.g. VW'),
      textInputAction: TextInputAction.next,
      textCapitalization: TextCapitalization.words,
      validator: (v) => (v == null || v.trim().isEmpty) ? 'Make is required' : null,
    ),
    const SizedBox(height: 12),
    TextFormField(
      controller: _yearCtrl,
      decoration: const InputDecoration(labelText: 'Year'),
      keyboardType: TextInputType.number,
      inputFormatters: [
        FilteringTextInputFormatter.digitsOnly,
        LengthLimitingTextInputFormatter(4),
      ],
      validator: (v) {
        final y = int.tryParse(v ?? '');
        if (y == null) return 'Enter a 4-digit year';
        if (y < 1990 || y > DateTime.now().year) {
          return 'Year must be 1990-${DateTime.now().year}';
        }
        return null;
      },
    ),
    const SizedBox(height: 12),
    DropdownButtonFormField<Cover>(
      initialValue: _cover,
      decoration: const InputDecoration(labelText: 'Cover'),
      items: [
        for (final c in Cover.values) DropdownMenuItem(value: c, child: Text(c.label)),
      ],
      onChanged: (c) => setState(() => _cover = c ?? _cover),
    ),
    const SizedBox(height: 12),
    _LicenceDateField(
      value: _licenceDate,
      onChanged: (d) => setState(() => _licenceDate = d),
    ),
    const SizedBox(height: 24),
    FilledButton.icon(
      onPressed: _submit,
      icon: const Icon(Icons.calculate),
      label: const Text('Get quote'),
    ),
  ],
);
```

Two things break this code if you miss them:

Import `services.dart`. `FilteringTextInputFormatter` and `LengthLimitingTextInputFormatter` live there, not in `material.dart`. Missing it gives *"Undefined name FilteringTextInputFormatter"*.

`initialValue` versus `value` on the dropdown. `DropdownButtonFormField`'s `value` parameter was replaced by `initialValue` in Flutter 3.33. If your SDK is older and you get *"The named parameter 'initialValue' isn't defined"*, use `value:` instead, or run `flutter upgrade`.

5.3 A custom FormField: the date picker

```
class _LicenceDateField extends StatelessWidget {
  const _LicenceDateField({required this.value, required this.onChanged});
  final DateTime? value;
  final ValueChanged<DateTime?> onChanged;

  @override
  Widget build(BuildContext context) {
    final text = value == null
        ? 'Select date'
        : MaterialLocalizations.of(context).formatMediumDate(value!);
    return FormField<DateTime>(
      initialValue: value,
      validator: (d) => d == null ? 'Licence date is required' : null,
      builder: (state) => InputDecorator(
        decoration: InputDecoration(
          labelText: 'Licence issue date',
          errorText: state.errorText,
          suffixIcon: const Icon(Icons.calendar_today),
        ),
        child: InkWell(
          onTap: () async {
            final now = DateTime.now();
            final picked = await showDatePicker(
              context: context,
              initialDate: value ?? DateTime(now.year - 5),
              firstDate: DateTime(1950),
              lastDate: now,
            );
            if (picked != null) {
              state.didChange(picked);
              onChanged(picked);
            }
          },
          child: Text(text),
        ),
      ),
    );
  }
}
```

This is the pattern for **any** non text field that has to take part in `validate()`:

1. Wrap it in `FormField<T>`.
2. Call `state.didChange(value)` whenever the value changes.
3. Render it with `InputDecorator` so it looks like the `TextFormField`s around it.

`showDatePicker` returns `Future<DateTime?>` and gives `null` if the user cancels. It needs `MaterialLocalizations`, which `MaterialApp` provides, so calling it from a screen is fine. Calling it from a context **above** `MaterialApp` throws *"No MaterialLocalizations found"*.

`showTimePicker` has exactly the same shape.

5.4 Dialogs, bottom sheets and snackbars

```
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('Quote saved'), behavior: SnackBarBehavior.floating),
);

final confirmed = await showDialog<bool>(
  context: context,
  builder: (ctx) => AlertDialog(
    title: const Text('Discard changes?'),
    content: const Text('Your unsaved quote will be lost.'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(ctx, false), child: const Text('Cancel')),
      FilledButton(onPressed: () => Navigator.pop(ctx, true), child: const Text('Discard')),
    ],
  ),
);
if (!context.mounted) return;
if (confirmed == true) {
  // act on it
}

showModalBottomSheet(
  context: context,
  showDragHandle: true,
  builder: (ctx) => const CoverPickerSheet(),
);
```

Three APIs, one pattern. They all return `Future`s, so you `await`, and **after every await you check `context.mounted` before touching the context again**. The user may have navigated away while you waited. The lint `use_build_context_synchronously` flags you when you forget.

Inside the dialog builder, pop with the dialog's own `ctx`. Using the outer `context` pops the wrong route.

Never show a dialog from `build()`. Trigger it from a callback.

5.5 Other inputs, gestures and the keyboard

```
InkWell(onTap: () {}, borderRadius: BorderRadius.circular(12), child: card)
GestureDetector(onLongPress: () {}, onHorizontalDragEnd: (d) {}, child: card)

Switch(value: _extras, onChanged: (v) => setState(() => _extras = v))
SwitchListTile(
  title: const Text('Roadside assist'),
  value: _ra,
  onChanged: (v) => setState(() => _ra = v),
)
CheckboxListTile(
  title: const Text('Accept terms'),
  value: _ok,
  onChanged: (v) => setState(() => _ok = v ?? false),
)
Slider(
  value: _excess, min: 0, max: 10000, divisions: 20,
  label: 'R ${_excess.round()}',
  onChanged: (v) => setState(() => _excess = v),
)

FocusScope.of(context).unfocus();
```

`InkWell` needs a `Material` ancestor to paint its ripple. `Card` and `Scaffold` provide one. If you see no ripple, wrap it in `Material(color: Colors.transparent, child: ...)`.

`GestureDetector` gives you raw gestures and no visual feedback at all.

On Android the soft keyboard can cover the submit button. `Scaffold` resizes the body by default, so wrapping the form in a `SingleChildScrollView` lets the field scroll into view.

LAB 2.2 · The quote capture form · 45 min

Goal: a validated `CaptureForm` that hands a `QuoteRequest` up to `CaptureScreen`, which shows the premium in a SnackBar.

Everyone must finish steps 1 to 4. The rest is bonus.

Steps

1. Create `lib/features/quote/presentation/capture_form.dart` with `CaptureForm` from Part 5.1. Remember the imports and the `dispose()`.
2. Add the five fields from Part 5.2 and the `_LicenceDateField` from Part 5.3.
3. On submit: validate, build the `QuoteRequest`, call `widget.onSubmit(request)`.
4. In `capture_screen.dart` replace the placeholder `Card` with `CaptureForm(onSubmit: ...)` and show the premium with a SnackBar.
5. Add a **Reset** `OutlinedButton` that calls `_formKey.currentState!.reset()` and clears both controllers.
6. Test three cases: an empty submit shows four errors, the year 1985 shows the range error, valid input shows the premium.
7. Add a confirmation `AlertDialog` on Reset when any field is filled, and check `context.mounted` after the await.

8. `flutter analyze` clean, then commit.

Imports for `capture_form.dart`

This is the block people miss.

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:quote_app/features/quote/domain/quote_model.dart';
```

Step 4: the changed part of `capture_screen.dart`

Replace the placeholder body and add the SnackBar handler:

```
final body = Padding(
  padding: const EdgeInsets.all(16),
  child: CaptureForm(onSubmit: (r) => _showPremium(context, r)),
);
```

```
void _showPremium(BuildContext context, QuoteRequest r) {
  final premium = calculatePremium(r);
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Premium ${premium.rands}')),
  );
}
```

Add `_showPremium` as a method on `CaptureScreen`, and add this import at the top:

```
import 'package:quote_app/features/quote/presentation/capture_form.dart';
```

Because `body` and `header` are no longer compile time constants, remove the `const` keywords from the `Row`, `Column` and `SingleChildScrollView` in that builder. The analyser will tell you exactly which ones.

Step 8 commands

```
flutter analyze
```

```
git add .
```

```
git commit -m "Day 2: capture form"
```

Stretch

- Add `autovalidateMode: AutovalidateMode.onUserInteraction` after the first submit attempt, using a `bool` in the State.
- Replace the cover dropdown with the `GridView` of cover cards, keeping it inside the form via `FormField<Cover>`.
- Use the `intl` package for date formatting:

```
flutter pub add intl
```

```
DateFormat.yMMMd('en_ZA').format(date)
```

Part 6 · Theming and white-label

6.1 ThemeData and Material 3 colour roles

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0B2545)),
    useMaterial3: true,
    inputDecorationTheme: const InputDecorationThemeData(border: OutlineInputBorder()),
    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
    ),
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: const Color(0xFF0B2545),
      brightness: Brightness.dark,
    ),
    useMaterial3: true,
  ),
  themeMode: ThemeMode.system,
  home: const CaptureScreen(),
)
```

```
final cs = Theme.of(context).colorScheme;
final tt = Theme.of(context).textTheme;

Container(
  color: cs.primaryContainer,
  child: Text('Premium', style: tt.titleLarge?.copyWith(color: cs.onPrimaryContainer)),
)
```

One seed colour generates the entire tonal palette: `primary`, `onPrimary`, `primaryContainer`, `secondary`, `surface`, `error`, and their dark mode equivalents. That is what makes white-label cheap. Change the seed, everything follows.

Role	Use it for
`primary`	The brand colour on key components: filled buttons, the app bar, a FAB
`onPrimary`	Text and icons drawn **on top of** `primary`
`primaryContainer`	A softer standout fill: headers, chips, highlighted cards
`onPrimaryContainer`	Text on `primaryContainer`
`surface`	Page and card backgrounds
`error` / `onError`	Validation and destructive actions

Text styles have names too: `displayLarge` down through `titleMedium`, `bodyMedium`, `labelSmall`. Use the names.

> **The rule for this course.** If you type `Colors.blue` or `fontSize: 18` inside a widget, you have broken white-label.

SDK note. On Flutter 3.35 and later the component theme types gained a `Data` suffix: `InputDecorationThemeData`, `AppBarThemeData`, `BottomAppBarThemeData`. The old names still compile, because `ThemeData` accepts both, but the `Data` names are current. If you are on an older SDK and get *"InputDecorationThemeData isn't defined"*, drop the `Data`.

6.2 BrandTheme

```
// lib/core/theme/brand_theme.dart
import 'package:flutter/material.dart';

class BrandTheme {
  const BrandTheme({
    required this.key,
    required this.name,
    required this.seed,
    required this.logoAsset,
    this.fontFamily,
  });

  final String key;
  final String name;
  final Color seed;
  final String logoAsset;
  final String? fontFamily;

  ThemeData toThemeData(Brightness brightness) {
    final scheme = ColorScheme.fromSeed(seedColor: seed, brightness: brightness);
    return ThemeData(
      colorScheme: scheme,
      useMaterial3: true,
      fontFamily: fontFamily,
      appBarTheme: AppBarThemeData(
        backgroundColor: scheme.primary,
        foregroundColor: scheme.onPrimary,
      ),
      inputDecorationTheme: const InputDecorationThemeData(border: OutlineInputBorder()),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
      ),
    );
  }
}

const brands = <String, BrandTheme>{
  'alpha': BrandTheme(
    key: 'alpha',
    name: 'Alpha Insure',
    seed: Color(0xFF0B2545),
    logoAsset: 'assets/brands/alpha.png',
  ),
  'beta': BrandTheme(
    key: 'beta',
    name: 'Beta Cover',
    seed: Color(0xFF8B1E3F),
    logoAsset: 'assets/brands/beta.png',
  ),
};
```

One immutable value object per brand. The component themes live inside `toThemeData`, so every brand shares the same **shape**, outlined inputs and 48 px buttons, and differs only in colour, name, logo and font.

Nothing in your widgets knows a brand exists. They read `Theme.of(context)`. Only `app.dart` knows which `BrandTheme` is active.

This same class carries through the week. Day 3 puts the key in a Riverpod provider. Day 5 sets it at build time from a flavour, with zero UI changes.

6.3 Switching brands at runtime

```
// lib/app.dart
import 'package:flutter/material.dart';
import 'package:quote_app/core/theme/brand_theme.dart';
import 'package:quote_app/features/quote/presentation/capture_screen.dart';

class QuoteApp extends StatefulWidget {
  const QuoteApp({super.key});

  @override
  State<QuoteApp> createState() => _QuoteAppState();
}

class _QuoteAppState extends State<QuoteApp> {
  String _brandKey = 'alpha';

  @override
  Widget build(BuildContext context) {
    final brand = brands[_brandKey]!;
    return MaterialApp(
      title: brand.name,
      theme: brand.toThemeData(Brightness.light),
      darkTheme: brand.toThemeData(Brightness.dark),
      themeMode: ThemeMode.system,
      home: CaptureScreen(
        brand: brand,
        onSwitchBrand: () => setState(
          () => _brandKey = _brandKey == 'alpha' ? 'beta' : 'alpha',
        ),
      ),
    );
  }
}
```

`QuoteApp` holds the key, `setState` swaps it, `MaterialApp` rebuilds with the new theme, and everything below recolours because every widget reads `Theme.of(context)`.

The `!` on `brands[_brandKey]!` is acceptable because the keys are constants we control. On Day 4, when the key could come from stored preferences, we guard it properly.

This pattern is deliberately naive: state at the root, a callback threaded down. Tomorrow's Riverpod refactor deletes the callback and is the worked example of lifting state.

6.4 Assets and fonts

```
flutter:
  uses-material-design: true
  assets:
    - assets/brands/
    - assets/images/hero.png
  fonts:
    - family: Inter
      fonts:
        - asset: assets/fonts/Inter-Regular.ttf
        - asset: assets/fonts/Inter-Bold.ttf
          weight: 700
```

Then:

```
flutter pub get
```

and **hot restart**, not hot reload. Assets are bundled at build time.

Two mistakes cause almost every *"Unable to load asset"*:

1. **YAML indentation.** `assets:` must sit under `flutter:` with exactly two spaces. Tabs are not allowed in YAML at all.
2. **No hot restart** after adding the file.

A folder entry such as `assets/brands/` includes that folder's direct children only. It is not recursive.

Asset paths always use forward slashes, including on Windows.

LAB 2.3 · Two brands, one app · 35 min

Goal: two brands, a runtime toggle, dark mode, and a logo per brand, with nothing hard-coded in any widget.

Steps

1. Create `lib/core/theme/brand_theme.dart` from Part 6.2. Pick your own two seed colours.
2. Rewrite `lib/app.dart` as the Stateful version from Part 6.3.
3. Give `CaptureScreen` two new required parameters, `brand` and `onSwitchBrand`, add the swap `IconButton` to the `AppBar`, and pass `brand.name` into `_BrandHeader`.
4. Search your widget files for hard-coded colours and font sizes, and replace every one with a theme role or a `textTheme` style.
5. Add `assets/brands/alpha.png` and `assets/brands/beta.png`, any two PNGs. Declare `assets/brands/` in `pubspec.yaml`, run `flutter pub get`, **hot restart**, and show the logo in the header with `Image.asset(brand.logoAsset)`.
6. Switch the emulator to dark mode and confirm both brands are still readable.
7. Commit.

Step 3: the changed part of `capture_screen.dart`

```
class CaptureScreen extends StatelessWidget {
  const CaptureScreen({super.key, required this.brand, required this.onSwitchBrand});
  final BrandTheme brand;
  final VoidCallback onSwitchBrand;
```

```
      appBar: AppBar(
        title: Text(brand.name),
        actions: [
          IconButton(
            icon: const Icon(Icons.swap_horiz),
            tooltip: 'Switch brand',
            onPressed: onSwitchBrand,
          ),
        ],
      ),
```

Add the import:

```
import 'package:quote_app/core/theme/brand_theme.dart';
```

Change the header line inside the `LayoutBuilder` to pass the brand through, and drop the `const` from it:

```
final header = _BrandHeader(name: brand.name, logoAsset: brand.logoAsset);
```

Then give `_BrandHeader` the second parameter and show the logo. Change its constructor and fields to:

```
  const _BrandHeader({required this.name, required this.logoAsset});
  final String name;
  final String logoAsset;
```

and replace the title `Text` inside the positioned `Column` with a `Row` that carries the logo:

```
Row(
  children: [
    Image.asset(logoAsset, height: 28, semanticLabel: '$name logo'),
    const SizedBox(width: 8),
    Expanded(
      child: Text(name,
          style: tt.titleLarge?.copyWith(color: cs.onPrimaryContainer)),
    ),
  ],
),
```

Do step 5 before you run this, or `Image.asset` throws because the file is not bundled yet.

Step 4: the search command

Windows PowerShell:

```
Select-String -Path lib\*.dart -Recurse -Pattern "Colors\.|fontSize:"
```

macOS:

```
grep -rn "Colors\.\|fontSize:" lib
```

`Colors.white24` on the watermark icon is the one acceptable exception, because it is a translucent overlay rather than a brand colour. Everything else should become a role.

Step 7 commands

```
flutter analyze
```

```
git add .
```

```
git commit -m "Day 2: brand theme"
```

Dark mode on the emulator

Settings, then Display, then Dark theme. Or from a terminal:

```
adb shell "cmd uimode night yes"
```

```
adb shell "cmd uimode night no"
```

Stretch

- Add a per brand `fontFamily` with a bundled `.ttf` from fonts.google.com.
- **Add a third brand without touching a single widget file.** If you can, the white-label design works.

Part 7 · Cheat sheets

Picking a layout widget

You want	Use
Things in a line	`Row`, `Column`
One child takes the leftover space	`Expanded`
One child takes up to its share	`Flexible`
A fixed gap	`SizedBox(height: 12)`
A flexible gap	`Spacer()`
Things layered on top of each other	`Stack` plus `Positioned`
Things that wrap onto a new line	`Wrap`
A responsive grid	`GridView.builder` with `SliverGridDelegateWithMaxCrossAxisExtent`
A long scrolling list	`ListView.builder`
A scrolling page of mixed content	`SingleChildScrollView` with a `Column`
Different layouts by width	`LayoutBuilder`
Avoid the notch	`SafeArea`
Force a size	`SizedBox`
Min or max bounds	`ConstrainedBox`
Position one child	`Align`, or `Center` for the middle

Coming from CSS

CSS	Flutter
`display: flex; flex-direction: row`	`Row`
`justify-content`	`mainAxisAlignment`
`align-items`	`crossAxisAlignment`
`flex: 1`	`Expanded`
`flex-wrap: wrap`	`Wrap`
`position: relative` and `absolute`	`Stack` and `Positioned`
`padding`	`Padding` or `Container(padding:)`
`margin`	`Container(margin:)` or a `SizedBox` between children
`@media (min-width: 700px)`	`LayoutBuilder`
CSS variables for theming	`ThemeData` and `ColorScheme`

Coming from React Native

React Native	Flutter
`<View>`	`Container`, `Column`, `Row`
`<Text>`	`Text`
`<FlatList>`	`ListView.builder`
`<TouchableOpacity>`	`InkWell` or `GestureDetector`
`<TextInput>`	`TextFormField`
`StyleSheet.create`	`ThemeData` plus widget properties
`Dimensions.get('window')`	`MediaQuery.sizeOf(context)`
`SafeAreaView`	`SafeArea`
`useState`	`setState` in a `StatefulWidget`

Part 8 · Troubleshooting

Symptom	Cause	Fix
Yellow and black stripes, "RenderFlex overflowed"	Child bigger than the Row or Column	Wrap the child in `Expanded` or `Flexible`, or wrap the flex in `SingleChildScrollView`
"Vertical viewport was given unbounded height"	`ListView` inside a `Column`	`Expanded(child: ListView(...))`. `shrinkWrap: true` only for tiny lists
"Incorrect use of ParentDataWidget"	`Expanded` not directly inside a Flex	Move the `Expanded` inside the `Row` or `Column`
"BoxConstraints forces an infinite width"	`TextField` inside a `Row`	`Expanded(child: TextField())`
Container is invisible	Loose parent, no child, no size	Give it a width and height, or a child
Stack has zero size	Only `Positioned` children	Give the Stack a sized child, or `fit: StackFit.expand`
"Unable to load asset"	pubspec indentation, path typo, or no restart	Two spaces under `flutter:`, `flutter pub get`, hot **restart**
"No MaterialLocalizations found"	Picker called from above `MaterialApp`	Call it from a screen's context
"A dismissed Dismissible widget is still part of the tree"	Item not removed in `onDismissed`	Remove it from the list in `onDismissed`, and use `ValueKey(id)`
"Undefined name FilteringTextInputFormatter"	Missing import	`import 'package:flutter/services.dart';`
"The named parameter 'initialValue' isn't defined"	SDK older than 3.33	Use `value:` instead, or `flutter upgrade`
"A TextEditingController was used after being disposed"	Controller created in `build()`	Create it as a field on the State
No ripple on `InkWell`	No `Material` ancestor	Wrap in `Material(color: Colors.transparent, child: ...)`
Keyboard covers the button	Body is not scrollable	`SingleChildScrollView` around the form
Emulator will not rotate	Auto rotate off	Emulator quick settings, or `Ctrl+Left` on Windows and `Cmd+Left` on macOS

If the emulator dies

Everything today works in a browser. Nothing we build needs a native plugin.

```
flutter run -d chrome
```

Resize the browser window to demonstrate the responsive breakpoint.

Part 9 · Homework for Day 3

Day 3 is state management, which the pre-course survey scored lowest. Twenty minutes of reading tonight is half the fix.

Read:

1. docs.flutter.dev/data-and-backend/state-mgmt/intro - the intro, "Ephemeral vs app state", and "Simple app state management"
2. riverpod.dev/docs/introduction/why_riverpod

Come with an answer to these four:

1. Define ephemeral state and app state in one sentence each. Then classify: the text in a form field, the current brand, the last quote, the emulator's dark mode setting.
2. What problem does `InheritedWidget` solve, and what does Provider add on top of it?
3. Riverpod lists the reasons it replaced Provider. Pick the two that matter most for an insurance app and say why.
4. Where does today's brand toggle belong, according to these docs?

Two of you will present answers 1 and 3 at the whiteboard at 09:00. We then refactor the brand toggle live using your answers.

Links from today

Topic	Link
Understanding constraints	docs.flutter.dev/ui/layout/constraints
Layouts in Flutter	docs.flutter.dev/ui/layout
Layout widget catalogue	docs.flutter.dev/ui/widgets/layout
Adaptive and responsive	docs.flutter.dev/ui/adaptive-responsive
Long lists	docs.flutter.dev/cookbook/lists/long-lists
Grid lists	docs.flutter.dev/cookbook/lists/grid-lists
Form validation	docs.flutter.dev/cookbook/forms/validation
Retrieve text field input	docs.flutter.dev/cookbook/forms/retrieve-input
Snackbars	docs.flutter.dev/cookbook/design/snackbars
Themes	docs.flutter.dev/cookbook/design/themes
Material 3 colour roles	m3.material.io/styles/color/roles
Assets and images	docs.flutter.dev/ui/assets/assets-and-images
Fonts	docs.flutter.dev/cookbook/design/fonts
DevTools Layout Explorer	docs.flutter.dev/tools/devtools/inspector
Course repo	github.com/Clynton19860/flutter
